

Explainable Semantic Textual Similarity via Dissimilar Span Detection

Diego Miguel Lozano^{1,†}, Daryna Dementieva^{1,2}, Alexander Fraser^{1,2}

¹ School of Computation, Information and Technology, Technical University of Munich (TUM)

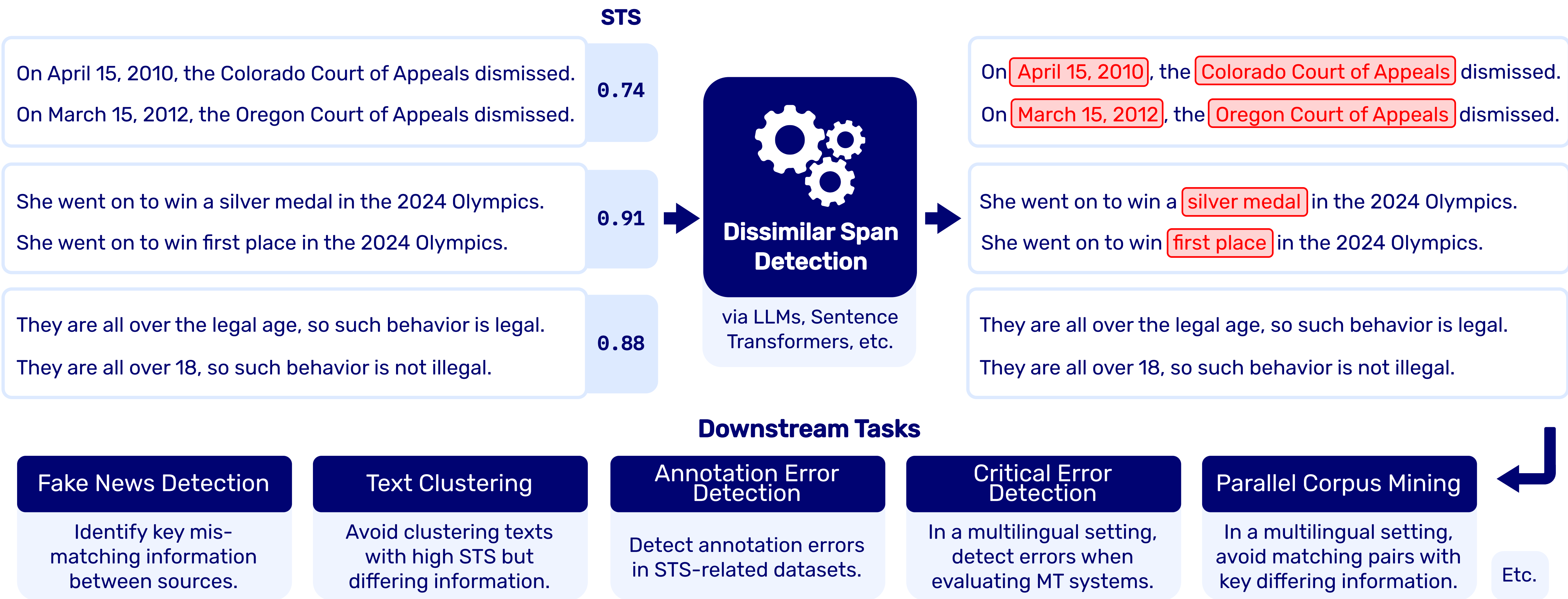
² Munich Center for Machine Learning (MCML)

[†] Currently affiliated to ELLIS Alicante



Cosine similarity is not always enough!

Outputting a single, non-interpretable number can mask fundamental differences between the texts being compared. We introduce the task of **Dissimilar Span Detection (DSD)**: Given two texts, identifying spans pairs with a common semantic function, but differing meanings.



1. Span Similarity Dataset (SSD)

New dataset (1,000 samples) crafted specifically for the task of DSD:

Sentence 1	Sentence 2	Span Similarity	Sentence Similarity
It was {{restored}} in the {{1980s}} .	It was {{destroyed}} in the {{eighties}} .	0, 1	0
Thank you for {{wasting my money}} .	Thank you for {{misusing my funds}} .	1	1
There are depots at {{Quilpie}} and {{Roma}} .	There are depots at {{Brisbane}} and {{Sydney}} .	0, 0	0

Table 1: Examples of shorter sentences from the SSD. Spans are denoted using double curly braces, and then annotated respectively with a 0, in case the span pair differs in meaning, or 1, if they are equivalent. We also annotate the sentence similarity of the pair as a whole.

Created in a semi-automatic way in two steps: (1) We let an LLM replace the spans, and (2) we manually review them and annotate the similarity. To ensure the correctness of the spans, six annotators manually annotate spans in 100 samples, achieving good inter-annotator and annotator-dataset agreement ($\kappa \in [0.61, 0.91]$).

3. Results

Method	Model Size	SSD				SemEval-2016	
		F1-Global	F1-NoDiff	F1-Diff	Time	F1-Diff	Time
LIME all-mpnet-base-v2	109M	0.463	0.782	0.223	1981.81	0.109	199.89
SHAP all-MiniLM-L6-v2	22.7M	0.366	0.434	0.131	44.47	0.306	4.52
Embedding all-mpnet-base-v2	109M	0.469	0.529	0.423	11.28	0.352	1.13
Embedding text-embedding-004	Billions	0.547	0.666	0.459	611.42	0.338	34.57
LLM Claude 3.5 Sonnet	Billions	0.750	0.895	0.640	337.80	0.389	39.22
Token Classification roberta-base	125M	0.690	0.838	0.574	1.23	0.441	0.10

Table 2: Summary of the results. Evaluation times are reported in minutes. For experiments relying on external APIs (text-embedding-004 and Claude 3.5 Sonnet), times might vary depending on rate limits, connection speeds, etc.

3.1 Downstream Task – Paraphrase Detection

Model	STS	STS + DSD
all-MiniLM-L6-v2	0.720	0.808
all-mpnet-base-v2	0.795	0.868

Table 3: Comparison of accuracies on the PAWS-Wiki Labeled using uniquely STS or combining STS with DSD.



DSD improves the performance on the task with no fine-tuning.

2. Experimental Setup

We propose 5 methods to tackle the task of DSD:

SHAP-DSD*	Based on the SHAP framework [1]. We calculate the dissimilarity score as: $\text{DissimilarityScore}(a, b) = 1 - \text{CosineSimilarity}(a, b)$ Tokens with dissimilarity above a threshold are considered as dissimilar.
LIME-DSD*	Similar to SHAP-DSD, but with the explanation weights predicted by LIME [2].
LLM-DSD	Uses an LLM to detect the dissimilar spans.
Token-Classification-DSD	A model is fine-tuned to predict dissimilar spans.
Embedding-DSD*	This method constitutes a novel contribution .

We calculate all possible unigram, bigram, ..., n -gram replacements from the first sentence to the second. For example, for the following 2 sentences:

the bird flies fast over the hill
the car rides fast over the hill

If we were considering the trigram ['the', 'bird', 'flies'], we would get the replacements:

1 the bird flies fast over the hill
2 the the bird flies over the hill
3 the car the bird flies the hill
4 the car rides the bird flies hill
5 the car rides fast the bird flies

We then calculate the *similarity gain* for each n -gram by comparing the resulting cosine similarity to the *base similarity* (i.e., the cosine similarity between the original sentences). Each unigram is assigned the gains coming from each of the n -grams in which it appears. These are then combined through an *aggregation function*:

$$\text{AggrGain}_{\text{unigram}} = \frac{1}{n} \cdot \sum_{i=1}^n \frac{\text{gains}_i}{i}$$

Unigrams with a gain above a threshold are considered as dissimilar.

* Methods that require a threshold. We find the optimal threshold by evaluating on the validation split.

Want to know more?



dmills.github.io/
dissimilar-span-detection